ALEX LANGUAGE

APPRENDRE ALEX DE ZÉRO

Parcours pratique pour débutants

Des premiers affichages aux projets structurés

Présenté par Alexandre Koffi • Version pédagogique 1.3

Le parcours en un coup d’œil

Une progression en étapes courtes, avec de la pratique à chaque leçon.

01 **Fondamentaux**

Affichage, variables, calculs

02 **Logique**

Conditions, boucles, listes

03 **Réutilisation**

Fonctions, chaînes, modules

04 **Code robuste**

Erreurs, classes, fichiers, tests

05 **Intermédiaire**

Types, compréhensions, lambdas

06 **Applications**

JSON, CLI, asynchrone, HTTP

Comment utiliser ce diaporama

Le but n’est pas de mémoriser : c’est d’écrire, d’exécuter et de tester.

- Enregistrez chaque exemple dans un fichier portant l’extension .al.
- Tapez vous-même le code au lieu de le copier sans le comprendre.
- Lancez le programme depuis la racine du projet ALEX.
- Si une commande échoue, vérifiez le dossier courant, le fichier et les modules disponibles.

EXEMPLE ALEX

```
print("Bonjour, monde !")
print("Je découvre ALEX.")
```

Commande indicative du guide : alexc run bonjour.al — adaptez-la à votre installation.

1. Votre premier programme

print(...) affiche un résultat dans le terminal.

- Un programme est une suite d’instructions.
- Le texte affiché doit être placé entre guillemets.
- Essayez plusieurs messages et observez le résultat.

EXEMPLE ALEX

```
print("Bonjour, monde !")
print("Je découvre le langage ALEX.")
```

À pratiquer : affichez votre prénom puis trois phrases différentes.

2. Variables et valeurs

Une variable conserve une valeur réutilisable.

- Utilisez := pour initialiser une variable.
- Utilisez = pour modifier une variable déjà créée.
- ALEX peut manipuler du texte, des nombres, des booléens et des listes.

EXEMPLE ALEX

```
nom := "Awa"
age := 18
actif := true
age = age + 1
print(nom, age, actif)
```

À pratiquer : créez produit, prix et stock, puis augmentez le stock de 5.

3. Calculs et comparaisons

Les opérateurs permettent de calculer et de prendre des décisions.

- + addition • - soustraction
- * multiplication • / division
- % reste de division
- >, <, == et != comparent deux valeurs.

EXEMPLE ALEX

```
a := 10
b := 3
print(a + b)
print(a % b)
print(a > b)
print(a == b)
```

À pratiquer : calculez le prix de 4 articles à 250 FCFA chacun.

4. Conditions

if et else permettent d’exécuter un bloc selon une condition.

- Une condition produit un résultat vrai ou faux.
- L’indentation définit les blocs d’instructions.
- Gardez une indentation régulière et évitez de mélanger tabulations et espaces.

EXEMPLE ALEX

```
note := 14
if note >= 10:
    print("Admis")
else:
    print("À reprendre")
```

À pratiquer : affichez « majeur » si age >= 18, sinon « mineur ».

5. Boucles et listes

Une boucle répète une action sur chaque élément.

- Une liste rassemble plusieurs valeurs.
- for parcourt les éléments d’une collection.
- Une variable total peut accumuler un résultat.
- Testez les listes avec peu d’éléments avant de les agrandir.

EXEMPLE ALEX

```
fruits := ["mangue", "banane", "orange"]
for fruit in fruits:
    print(fruit)

nombres := [3, 5, 7]
total := 0
for n in nombres:
    total += n
print(total)
```

À pratiquer : parcourez cinq prix et calculez leur somme.

6. Fonctions

Une fonction regroupe du code réutilisable.

- func introduit une fonction.
- Les paramètres sont les données reçues.
- return renvoie un résultat au code appelant.
- Une fonction peut afficher une information et/ou renvoyer une valeur.

EXEMPLE ALEX

```
func carre(n):
    return n * n

func saluer(nom):
    print("Bonjour " + nom)

saluer("Mariam")
print(carre(6))
```

À pratiquer : créez double(n), qui renvoie n * 2.

7. Chaînes et f-strings

Les chaînes servent à construire des messages lisibles.

- print peut afficher plusieurs valeurs.
- L’opérateur + permet de joindre du texte.
- Une f-string insère des valeurs entre accolades.

EXEMPLE ALEX

```
prenom := "Koffi"
age := 20
print("Prénom :", prenom)
print(f"{prenom} a {age} ans.")
```

À pratiquer : présentez un produit, son prix et sa quantité.

8. Utiliser un module

Les modules regroupent des fonctionnalités réutilisables.

- use charge un module documenté.
- Le guide présente math et json comme exemples.
- La disponibilité dépend de la version et de l’environnement.
- Vérifiez la bibliothèque standard avant de supposer qu’un module existe.

EXEMPLE ALEX

```
use math
print(math.abs(-7))
print(math.max(4, 9))
```

À pratiquer : testez plusieurs fonctions mathématiques disponibles.

9. Gérer une erreur

Prévoir les erreurs rend un programme plus fiable.

- try délimite une opération à surveiller.
- catch reçoit l’erreur rencontrée.
- Affichez un message utile pour aider au diagnostic.
- Le comportement exact dépend de l’opération et du runtime.

EXEMPLE ALEX

```
try:
    valeur := 10 / 0
    print(valeur)
catch e:
    print("Une erreur a été rencontrée :")
    print(e)
```

À pratiquer : protégez une opération potentiellement invalide.

10. Classes et objets

Une classe décrit des données et des méthodes ; un objet en est une instance.

- Une méthode est une fonction définie dans une classe.
- self désigne l’objet courant dans les exemples du guide.
- Commencez par maîtriser variables et fonctions avant les classes.

EXEMPLE ALEX

```
class Compteur:
    valeur := 0

    func init(depart):
        self.valeur = depart

    func ajouter(n):
        self.valeur = self.valeur + n
        return self.valeur

c := Compteur(5)
print(c.ajouter(3))
```

11. Fichiers : avancer avec la documentation

Pour les fichiers, ne devinez pas les noms des fonctions.

- Repérez le module file et sa documentation.
- Trouvez un exemple de lecture et un exemple d’écriture.
- Reproduisez d’abord l’exemple minimal.
- Prévoyez fichier absent, droits insuffisants et chemin incorrect.

Conseil du guide : vérifiez les API de la version installée.

12. Tester et déboguer

Vérifier le code avant de l’intégrer évite de propager des erreurs.

- run : exécuter un programme.
- check : vérification statique si disponible.
- test : lancer les tests pris en charge.
- fmt, doc, repl et debug dépendent de l’exécutable et de la version.

EXEMPLE ALEX

```
alexc check bonjour.al
alexc run bonjour.al
alexc test
```

Pour créer un projet, le guide cite : alexc new cli mon_app.

Projet 1 — Gestionnaire de notes

Réutilisez variables, listes, boucles, conditions et fonctions.

- Stocker les notes dans une liste.
- Calculer le total et la moyenne.
- Afficher admis si la moyenne atteint 10.
- Améliorer ensuite avec la meilleure et la plus faible note.

EXEMPLE ALEX

```
notes := [12, 15, 8, 10]
total := 0
for note in notes:
    total += note
moyenne := total / len(notes)
print(f"Moyenne : {moyenne}")
if moyenne >= 10:
    print("Résultat : admis")
else:
    print("Résultat : à reprendre")
```

13. Annotations de types

Les annotations précisent le type attendu pour une variable ou une fonction.

- Elles sont facultatives dans les exemples du guide.
- alexc check peut vérifier certains types et noms.
- Types présentés : int, float, bool, str, list[T], dict[K, V] et T?.

EXEMPLE ALEX

```
age: int := 19
nom: str := "Awa"
actif: bool := true

func doubler(n: int) -> int:
    return n * 2

print(doubler(age))
```

À pratiquer : annotez une fonction qui additionne deux entiers.

14. Compréhensions de listes

Créer une nouvelle liste avec une transformation ou un filtre.

- Transformez chaque élément d’une collection.
- Filtrez les valeurs selon une condition.
- La forme documentée inclut aussi les compréhensions de dictionnaires.

EXEMPLE ALEX

```
nombres := [1, 2, 3, 4, 5]
carres := [n * n for n in nombres]
pairs := [n for n in nombres if n % 2 == 0]
print(carres)
print(pairs)
```

À pratiquer : gardez uniquement les prix supérieurs à 1 000.

15. Lambdas et fermetures

Une lambda est une fonction courte définie par une expression.

- Utile pour une opération simple.
- Une fermeture peut conserver l’accès à une variable de son contexte.
- Testez les fonctions avec plusieurs valeurs.

EXEMPLE ALEX

```
carre := n => n * n
additionner := (a, b) => a + b
constante := () => 7

print(carre(5))
print(additionner(3, 4))
print(constante())
```

À pratiquer : créez une lambda qui multiplie un nombre par 10.

16. Manipuler le JSON

JSON permet de représenter des données structurées.

- parse transforme un texte JSON en valeur ALEX.
- stringify transforme une valeur en texte JSON.
- pretty produit une représentation lisible.
- load et save utilisent un chemin de fichier.

EXEMPLE ALEX

```
use json
texte := "{\"nom\": \"Clavier\", \"prix\": 5000}"
produit := json.parse(texte)
print(produit["nom"])
print(produit["prix"])
print(json.pretty(produit))
```

À pratiquer : créez un objet JSON contenant le nom d'un élève et sa moyenne.

17. Arguments de ligne de commande

Les arguments permettent de transmettre des données au lancement.

- argparse définit les arguments acceptés.
- Le guide présente parser, add_argument et parse_args.
- Les options exactes varient selon la version installée.

EXEMPLE ALEX

```
use argparse
p := argparse.parser(prog="salut", description="Exemple ALEX")
p.add_argument("nom", {"help": "nom de la personne"})
args := p.parse_args()
print("Bonjour", args["nom"])
```

Exemple de lancement dans le guide : alexc run salut.al Awa.

18. Premiers pas avec l’asynchrone

async/await permet d’attendre le résultat d’une tâche asynchrone.

- Les coroutines décrites coopèrent sur un seul thread.
- await attend le résultat d’une tâche.
- Vérifiez que asyncio et la syntaxe sont disponibles dans votre version.

EXEMPLE ALEX

```
use asyncio

async func calculer(n):
    await asyncio.sleep(0.1)
    return n * 2

async func main():
    a := calculer(10)
    b := calculer(21)
    print(await a, await b)

await main()
```

À pratiquer : changez les valeurs calculées et affichez les résultats.

19. Premiers pas avec HTTP

Le réseau exige de prévoir les délais, erreurs et autorisations.

- Le projet documente des clients et serveurs HTTP.
- Choisissez un service autorisé pour vos tests.
- Pour un serveur public, traitez authentification, validation, TLS et erreurs.
- Le guide recommande de consulter NETWORK.md.

EXEMPLE ALEX

```
use network.http_client as hc
client := hc.Client("https://api.exemple.org/", {}, 10.0)
reponse := client.get("")
print(reponse)
```

L'hôte de l'exemple est illustratif : ce n'est pas une API de test garantie.

20. Vérifier et organiser un projet

Séparez les responsabilités et testez les cas limites.

- Vérifiez le code, exécutez-le et testez les cas importants.
- Séparez le point d'entrée, la logique métier et la documentation.
- Utilisez des noms explicites et des commentaires utiles.
- Testez liste vide, valeur nulle, texte inattendu et fichier absent.

EXEMPLE ALEX

```
mon_projet/
    main.al
    calculs.al
    README.md

alexc check mon_programme.al
alexc run mon_programme.al
alexc test
```

Vérifiez la syntaxe d'import et la recherche de modules de votre version.

Projet 2 — Carnet de produits

Un catalogue en mémoire pour pratiquer les structures de données et JSON.

- Afficher le nom et le prix de chaque produit.
- Ajouter une quantité et calculer le montant total.
- Ajouter une recherche par nom.
- Enregistrer en JSON après vérification des fonctions de fichier.

EXEMPLE ALEX

```
print(produit["nom"], ":", produit["prix"], "FCFA")
```

Ce mini-projet pédagogique ne remplace pas une base de données.

Votre feuille de route sur 8 semaines

Avancez progressivement, avec une séance de pratique régulière.

S1 Leçons 1–4

Affichage, variables, calculs, conditions

S2 Leçons 5–7

Boucles, listes, fonctions, chaînes

S3 Leçons 8–10

Modules, erreurs, objets

S4 Leçons 11–12

Fichiers, tests, mini-projet

S5 Leçons 13–14

Types, compréhensions

S6 Leçons 15–16

Lambdas, fermetures, JSON

S7 Leçons 17–18

Arguments CLI, asynchrone

S8 Leçons 19–20

HTTP, organisation, mini-projet

LE MEILLEUR CODE S'APPREND EN PRATIQUANT.

Écrire. Exécuter. Tester. Comprendre. Améliorer.

Gardez les exemples simples, consultez la documentation de votre version d'ALEX et construisez vos propres projets.

Alexandre Koffi • ALEX LANGUAGE